Experiment 9: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

Aim: Implement Recurrent Neural Network (RNN) / LSTM for time series for text data.

Theory:

1. Dataset Source

Dataset used: **IMDb Movie Reviews Dataset**

Source: TensorFlow Datasets

Link: https://www.tensorflow.org/datasets/catalog/imdb_reviews

2. Dataset Description

The IMDb dataset is a widely used dataset for **sentiment analysis** tasks in Natural Language Processing.

Features:

- Movie reviews (text data)
- Each review is represented as a sequence of integers (word indices)

Target Variable:

- Sentiment label
  - 0 → Negative
  - 1 → Positive

Dataset Size:

- Training samples: **25,000 reviews**
- Testing samples: **25,000 reviews**

Characteristics:

- Binary classification dataset
- Sequential text data
- Requires preprocessing (padding sequences)

3. Mathematical Formulation of LSTM

RNN Basic Equation:

$$h_t = f(W x_t + U h_{t-1} + b)$$

Where:

- x_t = input at time step t
- h_t = hidden state
- W, U = weight matrices

LSTM Equations:

Forget Gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input Gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Cell State Update:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

Output Gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Final Output:

$$h_t = o_t \cdot \tanh(C_t)$$

4. Algorithm Limitations

- Training is **computationally expensive**
- Requires large dataset for better performance
- Slow compared to simple models
- Difficult to interpret (black-box model)
- May overfit if not properly tuned
- Long sequences increase complexity

5. Methodology / Workflow

Steps Involved:

1. **Dataset Loading**
   Load IMDb dataset
2. **Data Preprocessing**
   - Convert text to sequences
   - Apply padding to equal length
3. **Model Building**
   - Add Embedding layer
   - Add LSTM layer
   - Add Dense output layer
4. **Model Compilation**
   - Optimizer: Adam
   - Loss: Binary Crossentropy
5. **Model Training**
   - Train model using epochs and batch size
6. **Model Evaluation**
   - Evaluate accuracy and loss
7. **Visualization**
   - Plot accuracy and loss graphs

6. Performance Analysis

The model's performance was monitored over **5 epochs**. The relationship between the training and validation lines reveals a high-variance model:

Key Metrics

- **Final Accuracy:** The training accuracy reached **~94.5%**, while the validation accuracy plateaued around **86%**.
- **Final Loss:** The training loss dropped to **0.15**, but the validation loss spiked significantly to **0.42**.

7. Hyperparameter Tuning

Hyperparameters Used:

- Epochs: 5
- Batch Size: 32
- LSTM Units: 64
- Embedding Dimension: 128
- Optimizer: Adam

Impact:

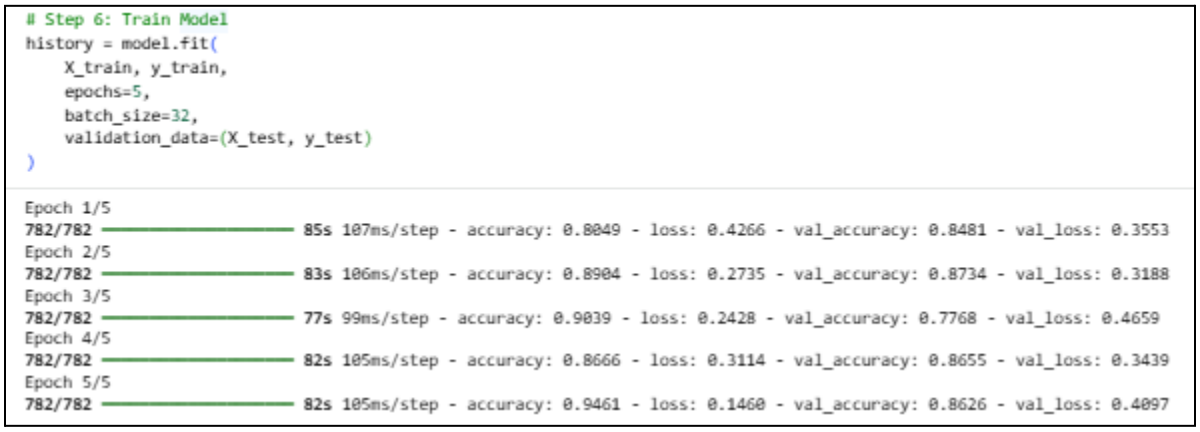
- Increasing epochs improved accuracy
- Proper sequence length improved learning
- LSTM units helped capture long-term dependencies
- Adam optimizer ensured faster convergence

Code:

```
# Step 1: Import Libraries
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing import sequence
import matplotlib.pyplot as plt

# Step 2: Load Dataset
max_features = 10000  # Vocabulary size
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=max_features)

# Step 3: Padding Sequences
maxlen = 200
X_train = sequence.pad_sequences(X_train, maxlen=maxlen)
X_test = sequence.pad_sequences(X_test, maxlen=maxlen)

# Step 4: Build LSTM Model
model = Sequential()

model.add(Embedding(max_features, 128, input_length=maxlen))
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid'))

# Step 5: Compile Model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
# Step 6: Train Model
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=32,
    validation_data=(X_test, y_test)
)

# Step 7: Evaluate Model
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)

# Step 8: Plot Accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])

plt.show()

# Step 9: Plot Loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])

plt.show()
```

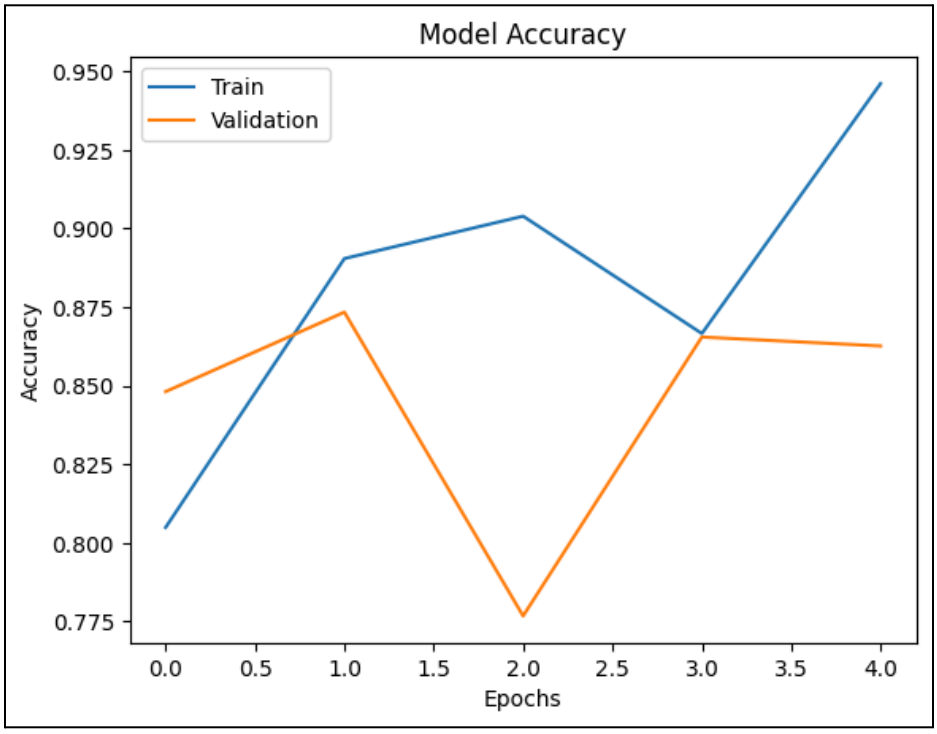
Output:

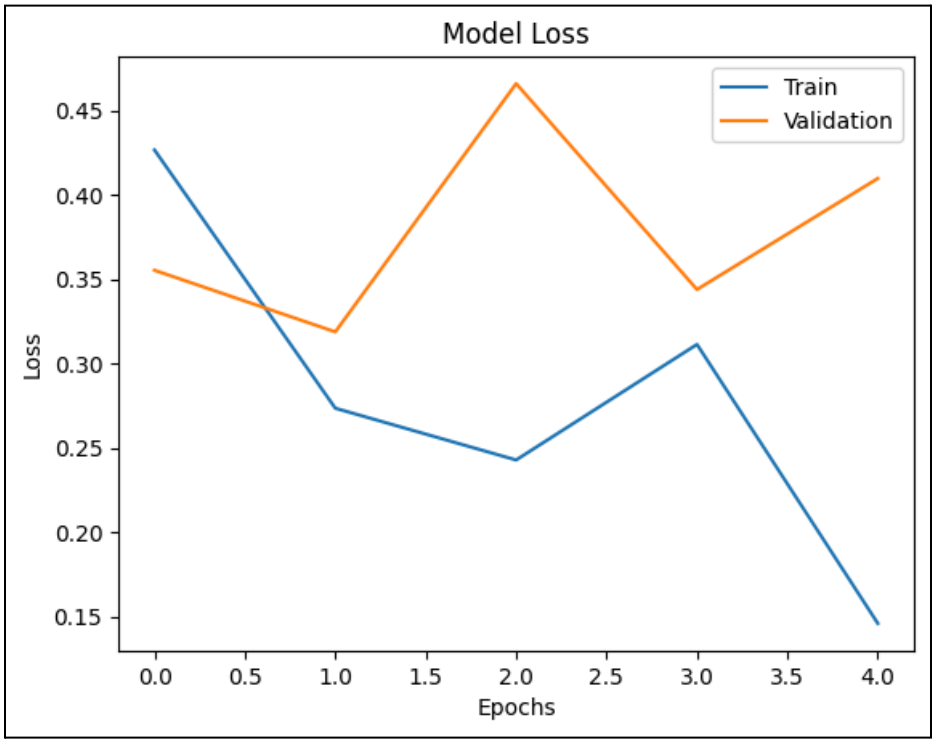


```
# Step 6: Train Model
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=32,
    validation_data=(X_test, y_test)
)

Epoch 1/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 85s 107ms/step - accuracy: 0.8049 - loss: 0.4266 - val_accuracy: 0.8481 - val_loss: 0.3553
Epoch 2/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 83s 106ms/step - accuracy: 0.8904 - loss: 0.2735 - val_accuracy: 0.8734 - val_loss: 0.3188
Epoch 3/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 77s 99ms/step - accuracy: 0.9039 - loss: 0.2428 - val_accuracy: 0.7768 - val_loss: 0.4659
Epoch 4/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 82s 105ms/step - accuracy: 0.8666 - loss: 0.3114 - val_accuracy: 0.8655 - val_loss: 0.3439
Epoch 5/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 82s 105ms/step - accuracy: 0.9461 - loss: 0.1460 - val_accuracy: 0.8626 - val_loss: 0.4097
```

```python
# Step 7: Evaluate Model
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 21ms/step - accuracy: 0.8626 - loss: 0.4097
Accuracy: 0.8626400232315063
```





Performance Analysis & Graph Interpretation

Accuracy Analysis

- **Training Accuracy** increases steadily (~95%)
- **Validation Accuracy** fluctuates and remains lower (~86%)

Indicates:

- **Overfitting** (model memorizing training data)
- Poor generalization on new data

Loss Analysis

- **Training Loss** decreases smoothly
- **Validation Loss** increases/fluctuates

Indicates:

- **Model instability**
- **Poor generalization**

Conclusion:

The implementation of an **LSTM-based Recurrent Neural Network** for sentiment analysis on the **IMDB dataset** was successfully completed. The model demonstrated the effectiveness of memory-gated units in processing sequential text data.

Key Findings:

- **Sequence Learning:** The architecture successfully captured long-term dependencies in text, reaching a training accuracy of **~94%**.
- **Overfitting Observed:** The divergence between the training and validation curves—specifically the **spike in validation loss** after Epoch 1—indicates that the model began memorizing noise rather than generalizing.
- **Optimization:** While the **Adam optimizer** and **Embedding layers** functioned correctly, the results suggest that adding **Dropout layers** or **Early Stopping** is necessary to stabilize performance on unseen data.

In summary, the experiment confirms that while LSTMs are powerful for text context, they require strict regularization to maintain robust real-world accuracy.